

Ace of SPADE: Selective and PARTial DEcryption with Post-Quantum Security

Anonymous Author(s)

Abstract

Functional Encryption (FE) is a privacy-preserving technique that allows computations over encrypted data. Specifically, the functional decryption algorithm does not output the plaintext data, but its evaluation under a specific function chosen by a third-party user. FE constructions suffer from a lack of versatility as they only support standard functions such as addition, inner product, and quadratic functions. While this functionality is important, it only allows quantitative analysis on encrypted data. This limitation was recently addressed by SPADE – a novel construction based on ElGamal that provides a way to perform qualitative analysis in a privacy-preserving way. Inspired by that paper, we designed Ace of SPADE (AoS) – a new, lattice-based, construction that is based on Ring-LWE and is quantum secure. Our scheme introduces partial decryption in a dynamic multi-client setting, enabling flexible user registration and fine-grained access control. In addition, AoS supports the important feature of ciphertext updatability and extends FE towards qualitative data analysis. Finally, we provide a formal security analysis, which is coupled with extensive experimental results that show its practicality.

Keywords

Functional Encryption, Ring-Learning With Errors, Partial Decryption, Post-Quantum Cryptography

ACM Reference Format:

Anonymous Author(s). 2026. Ace of SPADE: Selective and PARTial DEcryption with Post-Quantum Security. In . ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Nowadays, data privacy has become a major concern as personal and possibly sensitive information is being stored and processed online. Traditional and widely-used cryptographic techniques, such as Public-Key Encryption (PKE) provide strong confidentiality guarantees by ensuring that only authorized parties can decrypt data. However, those techniques are not currently enough to allow more complex computations on encrypted data while preserving confidentiality. Functional Encryption (FE) addresses this limitation by enabling computations over ciphertexts while allowing fine-grained access to encrypted data.

FE was introduced by Boneh *et al.* [10] as a generalization of PKE. FE was defined and developed as a system built upon multiple

decryption keys. While ciphertexts depend on a common public key pk , FE uses a key generation algorithm that outputs decryption keys dk_f associated with a function f . In contrast to traditional cryptographic techniques, encrypted data $Enc(x)$ is not aimed to be decrypted: a functional decryption key dk_f grants access to a computation $f(x)$ directly performed during the decryption – thus keeping the actual value x private. Thanks to this original multi-key construction, FE permits fine-grained computations on sensitive data, deriving from one or multiple users. However, ciphertexts are encrypted with a single master public key accessible to any party of the system. Some system models require users to have their own encryption keys, thus giving them more control over data sharing and processing. This problem can be solved through Multi-Client Functional Encryption (MCFE). MCFE uses *labels* during encryption and generates ciphertexts that can only be decrypted using decryption keys associated with the same label. Despite significant progress in the field, FE still fails to handle complex functions. Indeed, most efficient constructions only support standard functions such as inner product [2, 3, 7] or quadratic polynomials [8, 27]. To address this gap, Nuoskala *et al.* [24] proposed an original construction, called SPADE, supporting *partial decryption*. In this construction, a user is provided with a decryption key dk_v , for a plaintext value v , and a ciphertext vector $c = Enc(x)$ whose underlying plaintext $x = (x_1, \dots, x_n)$ is hidden. Given these two elements, a client can run a decryption algorithm that performs an equality check and outputs whether $x_i == v$ for $i \in [n]$, but provides no information about the values of the other elements. What makes this functionality interesting is that, unlike existing FE schemes, SPADE can operate on *qualitative* data, which is incompatible with addition or multiplication. This is an advantage over typical FE constructions since it allows an analyst to perform qualitative analysis on encrypted data – therefore offering richer opportunities for capturing knowledge in a privacy-preserving manner.

This article presents Ace of SPADE (AoS), a post-quantum secure scheme based on the broadly-used Ring-Learning with Errors (RLWE) [19] hardness assumption. Introduced by Lyubashevsky *et al.* in 2012, RLWE is a variant of the Learning With Errors (LWE) problem which is a lattice-based problem that has proved resistant to known quantum attacks.

In addition to its core lattice-based MCFE construction, our scheme extends its range of applicability by incorporating a recent technique called *ciphertext updatability* [12] that allows the update of a label without re-encryption. Due to these elements, AoS surpasses the previous ElGamal-based SPADE construction both in terms of security and applicability.

1.1 Contributions

The main contributions of this paper can be summarized as follows:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

- C1. The first contribution of this paper is the creation of the scheme Ace of SPADE, the first quantum-secure FE construction that allows both quantitative and qualitative analysis through encrypted data.
- C2. We extend the functionality of the proposed scheme by adding a ciphertext update mechanism. This is quite an important addition since it is a feature that is mostly missing from the current state-of-the-art FE schemes.
- C3. Additionally, we show how Ace of SPADE could be used in online services that will be using our FE construction. To do so, we present a use-case protocol, that demonstrates the applicability of our construction in practical scenarios. This protocol is supported by a thorough security analysis, where we formally prove the security of both the FE construction and the protocol against the existence of a powerful adversary.

1.2 Organization

The paper is organized as follows. Section 2 reviews related works on FE, partial decryption, the challenge of performing qualitative analysis through FE, as well as the problem of ciphertext updatability. Section 3 provides the necessary background (i.e., definitions and notations) needed for this work. Section 4 introduces the AoS – a lattice-based and quantum-secure FE construction which consists of this work’s core contribution. Section 5 describes a use-case protocol illustrating how AoS and its ciphertext update mechanism can be used in practice. In Section 6 a formal security analysis of both the FE construction and the protocol is provided. Section 7 reports the benchmarking results for the protocol. Finally, Section 9 concludes the paper.

2 Related Work

Functional Encryption and Multi-Client Functional Encryption. FE allows a user to compute specific functions on encrypted data without revealing the underlying plaintext and only obtains the result of that specific function. For example, Inner Product Functional Encryption (IPFE) [3, 14, 25] and quadratic polynomials [8] being some of the most studied functions. The number of supported functions is still a growing field; we can add the recent discovery for *partial decryption* [24] in 2024.

In 2014 a new functionality to FE was introduced: *Multi-Client Functional Encryption* by Goldwasser [15] which grow popular and is now one of the main features one can expect from an FE scheme. In an MCFE scheme, a ciphertext depends on a specific label that limits the decryption to users owning a decryption key for this specific label. While this feature is perfect for a system requiring fine-grained access control, it narrows flexibility in data management. Indeed, over the past dozen years, countless constructions have been proposed [4, 17, 18], covering a large range of functionalities and system models. Another notable recent technique, *Ciphertext Updatability* [12], allows an authority to deliver update tokens with the ability to update the label under which a message was encrypted.

Ciphertext Updatability. The idea of rightfully tampering with a ciphertext has been introduced by Boneh *et al.* [9] in 2013. Named *Updatable Encryption* (UE), this technique consists of modifying a ciphertext to change, or update, the key it is encrypted with. Whilst

simple, this feature has sparked interest in the field [5, 20, 22, 26]. Its adaptation to MCFE has emerged with the article of Cini *et al.* [12], who defined the notion of Ciphertext Updatable Functional Encryption (CUFE). While classical UE updates the complete key, CUFE updates the label under which the ciphertext was encrypted, without having to re-encrypt it from scratch. To sum up, the owner of the model can decide which ciphertext’s lifetime can be extended for further use. This paradigm being very recent, little work has been conducted to propose CUFE schemes. However, we believe that this technique will gain more attention in the future and will soon be an expected feature in most MCFE schemes.

Partial Decryption. FE schemes allow to compute specific functions on ciphertexts. However, FE schemes that allow fine-grained control and selective computations over encrypted data, while revealing only a part of the plaintext, are difficult to come by.

To the best of our knowledge, **SPADE [24] is the only FE-style construction providing selective partial decryption through function-derived keys while preserving compatibility with qualitative analysis.** While there are other techniques for decrypting parts of an encrypted database, these rely either on identifying specific patterns in a collection of ciphertexts [16] or on permutations [6].

SPADE is an FE scheme introduced by Nuoskala *et al.* in [24]. In the article, the authors achieve fine-grained control over ciphertext data during decryption. It is considered state-of-the-art in FE partial decryption. It relies on Cyclic Groups and Additive ElGamal and can be applied to both quantitative and qualitative data. It served as the main inspiration for AoS. Despite its novelty, SPADE lacks additional components, such as ciphertext updates, as in [12], and post-quantum security.

3 Background

Notation. For a positive integer n , $[n]$ is the set $\{1, \dots, n\}$. Vectors are denoted by bold lowercase letters, and matrices by bold uppercase letters. We denote the modulus operation by $[x]_q = x \bmod q$. We denote $y \leftarrow \mathcal{Y}$ when y is chosen uniformly at random from a set $\mathcal{Y}$. We denote $\mathbb{Z}_q$ the set of integers in $(-q/2, q/2]$, $\mathbb{Z}_q[X]$ the set of polynomials in X and, $\mathcal{R}_q = \mathbb{Z}_q[X]/P(X)$ the quotient ring, if $P(X)$ is irreducible in $\mathbb{Z}_q[X]$. An algorithm $\mathcal{A}$ is said to be (probabilistic) polynomial time ((P)PT) if it is a randomized algorithm such that there exists a polynomial P such that for any input y , the running time of $\mathcal{A}(y)$ is bounded by $|P(y)|$. A function $f : \mathbb{N} \mapsto \mathbb{R}$ is negligible if $\forall \epsilon \in \mathbb{N}, \exists \epsilon_0 \in \mathbb{N}$ such that $\forall \epsilon \geq \epsilon_0, f(\epsilon) < \epsilon^{-\epsilon}$. An arbitrary negligible function is denoted $\text{negl}(\cdot)$. Two elements a, b are said to be indistinguishable, denoted $a \sim b$, if $\Pr[\beta' = \beta; c \leftarrow \beta \cdot a + (1 - \beta) \cdot b \mid \beta \leftarrow \{0, 1\}] = 1/2 + \text{negl}(\cdot)$ where β' is the adversary’s guess, and β a random bit.

3.1 Functional Encryption

Definition 3.1 (Functional Encryption (FE)). A functional encryption scheme FE for message space $\mathcal{M}$ and class of functions $\mathcal{F} \subseteq \{f : \mathcal{M} \rightarrow \mathcal{Y}\}$, consists of four PT algorithms Setup, KDer, Enc, Dec defined as follows:

- Setup (1^λ) . The setup algorithm is a probabilistic algorithm. On the input of a security parameter λ , it outputs a secret key sk and a public key pk ;

- $\text{Enc}(\text{pk}, x)$. The encryption algorithm is a probabilistic algorithm. On the input of the public key pk and a message x in the message space $\mathcal{M}$, it outputs a ciphertext c .
- $\text{KDer}(\text{sk}, f)$. The key derivation algorithm can be deterministic or probabilistic. On the input of the secret key sk and a function $f \in \mathcal{F}$, it outputs a decryption key dk_f .
- $\text{Dec}(\text{dk}_f, c)$. The decryption algorithm is a deterministic algorithm. On the input of a decryption key dk_f and a ciphertext c , it outputs an element $y \in \mathcal{Y}$.

Correctness: For all parameters $\lambda \in \mathbb{N}$, $(\text{sk}, \text{pk}) \leftarrow \text{Setup}(1^\lambda)$, $f \in \mathcal{F}$ and $x \in \mathcal{M}$,

$$\mathbb{P} \left[\text{Dec}(\text{dk}_f, \text{Enc}(\text{pk}, x)) \neq f(x) \mid \text{dk}_f \leftarrow \text{KDer}(\text{sk}, f) \right]$$

is negligible in λ .

A functional encryption scheme only supports single-input functions, operating over $\mathcal{M}$. In many scenarios, it is necessary to extend the set of functions $\mathcal{F}$ to multiple inputs, over vectors of plaintexts in $\mathcal{M}^m$. Additionally, as previously mentioned, supporting multiple clients (MCFE) requires specific encryption and a key derivation algorithm taking an additional private identifier α , as input.

Definition 3.2 (Multi-Client Functional Encryption (MCFE)). An m -multi-client functional encryption scheme MCFE for message space $\mathcal{M}$ and class of functions $\mathcal{F} \subseteq \{f : \mathcal{M}^m \rightarrow \mathcal{Y}\}$, consists of four PT algorithms Setup , Enc , KDer , Dec defined as follows:

- $\text{Setup}(1^\lambda, m)$. The setup algorithm is a probabilistic algorithm. On the input of a security parameter λ and an integer $m \in \mathbb{N}$, it outputs a set secret keys $\text{sk} = (\text{sk}_1, \dots, \text{sk}_m)$ and a set of public keys $\text{pk} = (\text{pk}_1, \dots, \text{pk}_m)$.
- $\text{Enc}(\text{pk}_i, x_i, \alpha_i)$. The encryption algorithm is a probabilistic algorithm. On the input of the i -th public key pk_i , a message $x_i \in \mathcal{M}$, and a private identifier α_i , it outputs a ciphertext c_i for $i \in [m]$.
- $\text{KDer}(\text{sk}_i, f, \alpha_i)$. The key derivation algorithm can be deterministic or probabilistic. On the input of the secret key sk , a function $f \in \mathcal{F}$ and a private identifier α_i , it outputs a decryption key $\text{dk}_{\alpha_i, f}$.
- $\text{Dec}(\text{dk}_{\alpha_i, f}, c)$. The decryption algorithm is a deterministic algorithm. On the input of a decryption key $\text{dk}_{\alpha_i, f}$ and a ciphertext vector $\mathbf{c} = (c_1, \dots, c_m)$, it outputs an element $y \in \mathcal{Y}$.

Correctness: For all parameters $\lambda, m \in \mathbb{N}$, $(\text{sk}, \text{pk}) \leftarrow \text{Setup}(1^\lambda, m)$, $f \in \mathcal{F}$, identifiers $\alpha_1, \dots, \alpha_m$ and message vector $\mathbf{x} = (x_1, \dots, x_m) \in \mathcal{M}^m$,

$$\mathbb{P} \left[\text{Dec}(\text{dk}_{\alpha_i, f}, \mathbf{c}) \neq f(\mathbf{x}) \mid \text{dk}_{\alpha_i, f} \leftarrow \text{KDer}(\text{sk}_i, f, \alpha_i) \right. \\ \left. \mathbf{c} \leftarrow \text{Enc}(\text{pk}_i, x_i, \alpha_i) \right] \quad (1)$$

is negligible if $\alpha_1 = \dots = \alpha_m$ and overwhelming otherwise.

Remark: For the sake of brevity and readability, we sometimes use the notation $\mathbf{c} \leftarrow \text{Enc}(\text{pk}, \mathbf{x}, \alpha)$ to denote the encryption of the full vector $\mathbf{x} = (x_1, \dots, x_m)$ into a vector of ciphertexts $\mathbf{c} = (c_1, \dots, c_m)$

for keys $\text{pk} = (\text{pk}_1, \dots, \text{pk}_m)$ and common identifier α . This notation is valid, since it corresponds to single-input FE for message space $\mathcal{M}^m \times \mathcal{X}$, where $\mathcal{X}$ would denote the space in which α lies.

Ciphertext Updatability is a feature allowing better management of the ciphertexts encrypted using labels δ . In short, it permits us to update the label under which a certain ciphertext was encrypted. To perform this update, an authority can generate a token $\text{tk}_{\delta \rightarrow \delta'}$ that can update any ciphertext encrypted with a label δ into a ciphertext encrypted with the label δ' .

Definition 3.3 (Ciphertext Updatability (CU)). Let MCFE be an MCFE scheme for message space $\mathcal{M}$ and class of functions $\mathcal{F} \subseteq \{f : \mathcal{M}^m \rightarrow \mathcal{Y}\}$. Let $(\text{TokGen}, \text{Upd})$ be a pair of algorithms defined as follows:

- $\text{TokGen}(\text{sk}, \delta, \delta')$. The token generation algorithm can be deterministic or probabilistic. On the input of the secret key sk and two labels δ, δ' , it outputs a token $\text{tk}_{\delta \rightarrow \delta'}$.
- $\text{Upd}(\text{tk}_{\delta \rightarrow \delta'}, c_\delta)$. The update algorithm is a probabilistic algorithm. On the input of a token $\text{tk}_{\delta \rightarrow \delta'}$ and a ciphertext vector c_δ , it outputs a ciphertext $c_{\delta'}$.

Correctness: For all parameters $\lambda, m \in \mathbb{N}$, $(\text{sk}, \text{pk}) \leftarrow \text{Setup}(1^\lambda, m)$, $f \in \mathcal{F}$, labels δ, δ' , message vector $\mathbf{x} = (x_1, \dots, x_m) \in \mathcal{M}^m$, ciphertext vector $\mathbf{c}_\delta$ where $c_{i, \delta} \leftarrow \text{Enc}(\text{pk}_i, x_i, \delta)$ and $\text{dk}_{\delta^*, f} \leftarrow \text{KDer}(\text{sk}, f, \delta^*)$, $\delta^* \in \{\delta, \delta'\}$, $\text{tk}_{\delta \rightarrow \delta'} \leftarrow \text{TokGen}(\text{sk}, \delta, \delta')$ and $\mathbf{c}_{\delta'} \leftarrow \text{Upd}(\text{tk}_{\delta \rightarrow \delta'}, \mathbf{c}_\delta)$ the following equality holds:

$$\text{Dec}(\text{dk}_{\delta', f}, \mathbf{c}_{\delta'}) = \text{Dec}(\text{dk}_{\delta, f}, \mathbf{c}_\delta)$$

with overwhelming probability.

3.2 Ring-Learning With Errors

Introduced by Lyubashevsky *et al.* [19], RLWE is a structured variant of the LWE problem that improves efficiency by operating over polynomial rings rather than integer vectors. While traditional LWE parameters are integers used to encrypt a single bit, RLWE operates over a polynomial ring $\mathcal{R}_q := \mathbb{Z}_q[X]/(P(X))$ where $P(X)$ is typically a cyclotomic polynomial. A common instantiation of RLWE uses the cyclotomic polynomial $P(X) = X^n + 1$, which is secure when n is a power of 2, ensuring that $X^n + 1$ is irreducible. It is a significant performance improvement, allowing a broader use of lattice-based encryption in PKE and FE, in scenarios where the use of standard LWE parameters would be impossible. This is particularly true for HE schemes [11, 13] whose evaluation algorithm can prove expensive in terms of computation.

Remark: For efficiency purposes, we restrict coefficients to small sets. We use the terms ternary for the coefficients $\{-1, 0, 1\}$, and binary for $\{0, 1\}$. Furthermore, by $\mathcal{R}_2$ we refer to polynomials with coefficients in $\{-1, 0, 1\}$.

Definition 3.4 (Ring-Learning With Errors (RLWE) distribution). Let the parameters be $(n, m, q, \mathcal{X}_{\text{sk}}, \mathcal{X}_e)$ where $n \geq 1$ is an integer, m is the number of samples, q is the modulus, $\mathcal{X}_{\text{sk}}$ is the secret distribution over $\mathbb{Z}[X]/(X^n + 1)$ and $\mathcal{X}_e$ is the error distribution over $\mathbb{Z}_q[X]/(X^n + 1)$.

For a secret $\text{sk}(x) \in \mathbb{Z}_q[X]/(X^n + 1)$ chosen according to $\mathcal{X}_{\text{sk}}$, the RLWE distribution samples $a(x) \in \mathbb{Z}_q[X]/(X^n + 1)$ uniformly

at random, samples $e(x) \in \mathbb{Z}_q[X]/(X^n + 1)$ from $\mathcal{X}_e$, computes $b(x) := a(x) \cdot \text{sk}(x) + e(x) \bmod q$ and outputs $(a(x), b(x))$.

Definition 3.5 (Search-RLWE and Decisional-RLWE problems). From an RLWE distribution, one defines the two following hard problems:

- RLWE Search problem: the Search RLWE problem asks to recover sk given m samples $\{(a, b_i = a \cdot \text{sk} + e_i) : i = 1 \dots m\}$ from the RLWE distribution.
- RLWE Decisional problem: the Decisional RLWE problem asks to decide whether samples $(a, b_i) : i = 1 \dots m$ are sampled from the RLWE distribution or are chosen uniformly at random from $\mathbb{Z}_q[X]/(X^n + 1) \times \mathbb{Z}_q[X]/(X^n + 1)$.

The Search-RLWE and Decisional-RLWE problems are proven hard, as standard hard lattice problems, with appropriately chosen parameters [19].

Definition 3.6 (RLWE-based Encryption). An RLWE-based encryption scheme for message space $\mathcal{M} = \mathcal{R}_t$, consists of three algorithms RLWE.Setup, RLWE.Enc, RLWE.Dec defined as follows:

- RLWE.Setup($1^\lambda, \text{pp}$). On the input of a security parameter λ , outputs a secret key sk and a public key pk , where $\text{sk} \leftarrow \mathbb{Z}_2[X]/(X^n + 1)$ and $\text{pk} = (b, a)$ for $a \leftarrow \mathcal{R}_q$ and $b = [-a \cdot \text{sk} + t \cdot e_0]_q$, with $e_0 \leftarrow \mathcal{D}_{q,\sigma}$;
- RLWE.Enc(pk, x). On the input of a public key pk and a message x in the message space, samples $u \leftarrow \mathbb{Z}_2[X]/(X^n + 1)$, $e_1, e_2 \leftarrow \mathcal{D}_{q,\sigma}$ and outputs a ciphertext $\mathbf{c} = (c_1, c_2)$, where $c_1 = b \cdot u + t \cdot x$ and $c_2 = a \cdot u + t \cdot e_2$;
- RLWE.Dec($\text{sk}, \mathbf{c}$). On the input of a decryption key sk and a ciphertext $\mathbf{c} = (c_1, c_2)$, computes $x^* \leftarrow c_1 + \text{sk} \cdot c_2$, and outputs the message $x \in \mathcal{M}$ obtained by rounding from x^* .

4 AoS Scheme – Instantiation From Lattices

This section describes the formal construction of Ace of SPADE (AoS). In Section 4.1 we provide the core algorithms (Setup, Enc, KDer, Dec) constituting the MCFE scheme. Then, in Section 4.2, we extend the core construction into a CUFÉ scheme, embedding AoS with a pair of algorithms (TokGen, Upd).

4.1 Core Construction

This section formally defines the partial decryption FE scheme we obtain from an RLWE scheme with public parameters $\text{pp} = (n, m, q, \mathcal{X}_{\text{sk}}, \mathcal{X}_e)$, as detailed in Definition 3.4. We parametrize pp by a security parameter λ , with $q = q(\lambda)$ such that $\log_2(\log_2 q) \in \mathbb{N}^*$, $\mathcal{X}_{\text{sk}}$ the uniform distribution over ternary polynomials in $\mathcal{R}$, denoted $\mathcal{R}_2 := \mathbb{Z}_2[X]/(X^n + 1)$, and $\mathcal{X}_e$ the Gaussian distribution in $\mathcal{D}_{\sigma,q}$ of standard derivation $\sigma = \sigma(\lambda)$ in $\mathcal{R}_q$. We denote $\mathcal{M} = \mathcal{R}_t = \mathbb{Z}_t[X]/(X^n + 1)$ the message space for a positive integer $t = t(\lambda)$, and $\mathcal{C} = \mathcal{R}_q \times \mathcal{R}_q$ the ciphertext space.

Setup. During the setup step in Algorithm 1, the secret key sk and the public key pk are generated as follows: sk is a ternary polynomial sampled randomly in $\mathbb{Z}_2[X]/(X^n + 1)$; a is a polynomial sampled randomly in $\mathbb{Z}_q[X]/(X^n + 1)$ and e_0 and e_α are sampled from $\mathcal{D}_{q,\sigma}$ and $\mathcal{D}_{q,\sigma}$, respectively. A identifier α is a ternary polynomial in $\mathbb{Z}_2[X]/(X^n + 1)$.

The public key pk is composed of m RLWE instances $\text{pk}_i = (b_i, a) \in \mathcal{R}_q^2$, where $b_i = [-a \cdot \text{sk}_i + t \cdot e_{0,i}]$, and a key $k_{\alpha_i} = a \cdot \alpha + t \cdot e_{\alpha_i}$ associated to the identifier α_i is a "helper" key.

Notations of α and α_i

In the algorithm Setup, we choose the α as a list of the polynomials storing the different α_i for each user. Meanwhile, for the algorithms Enc, KDer and Dec, we use the notation α_i to denote the private identifier of a single user u_i . However, in Section 6 we use α instead of α_i to light up the writing.

Algorithm 1 Setup($1^\lambda, \text{pp}$)

Input: Takes as input a security parameter λ and the public parameters pp .

Output: Outputs a secret key $\text{sk} = (\text{sk}_1, \dots, \text{sk}_m)$ and a public key $\text{pk} = (\text{pk}_1, \dots, \text{pk}_m)$ and the k_α key.

- 1: Sample $a \leftarrow \mathcal{R}_q$ and $\text{sk} = (\text{sk}_1, \dots, \text{sk}_m)$ where $i \in [m]$, $\text{sk}_i \leftarrow \mathcal{R}_2$
- 2: Sample $e_0 = (e_{0,1}, \dots, e_{0,m})$ where $i \in [m]$, $e_{0,i} \leftarrow \mathcal{D}_{q,\sigma}^m$ and $e_\alpha \leftarrow \mathcal{D}_{q,\sigma}$
- 3: Sample $ID = (\alpha_1, \dots, \alpha_m)$ where $i \in [m]$, $\alpha_i \leftarrow \mathcal{R}_2$ and set $k_{\alpha_i} \leftarrow [a \cdot \alpha_i + t \cdot e_{\alpha_i}]_q$
- 4: Set $\text{pk} = (\text{pk}_1, \dots, \text{pk}_m)$, where $i \in [m]$, $\text{pk}_i = (b_i, a) \in \mathcal{R}_q^2$, for $b_i \leftarrow [-a \cdot \text{sk}_i + t \cdot e_{0,i}]_q$
- 5: **return** $(\text{sk}, \text{pk}, k_{\alpha_i})$

Encryption. The encryption Algorithm 2 is a probabilistic algorithm. On the input of a message $x \in \mathbb{Z}_t[X]/(X^n + 1)$, the i -th public key $\text{pk}_i = (b_i, a) \in \mathcal{R}_q^2$ and a identifier α_i , outputs a ciphertext $\mathbf{c}_x = (c_1, c_2)$.

To process that part, we sample u uniformly at random in $\mathcal{R}_2$ and the errors e_1, e_2 according to the Gaussian distribution $\mathcal{D}_{q,\sigma}$. The first component of the ciphertext is computed as $c_1 = [b_i \cdot \alpha_i + t \cdot e_1 + u \cdot x]_q$ contains the message payload x , and its second part $c_2 = [a \cdot \alpha_i + t \cdot e_2 - u]_q$ is a helping information for the decryption.

Algorithm 2 Enc(pk_i, x, α_i)

Input: Takes as input the i -th public key pk_i and a plaintext $x \in \mathcal{R}_t$ and α_i a private identifier.

Output: Outputs the ciphertext $\mathbf{c}_x = (c_1, c_2)$

- 1: Sample $u \leftarrow \mathcal{R}_2$, and $e_1, e_2 \leftarrow \mathcal{D}_{q,\sigma}$
- 2: Set $c_1 \leftarrow [b_i \cdot \alpha_i + t \cdot e_1 + u \cdot x]_q$ and $c_2 \leftarrow [a \cdot \alpha_i + t \cdot e_2 - u]_q$
- 3: **return** $\mathbf{c}_x = (c_1, c_2)$

Key Derivation. The key derivation algorithm is a probabilistic algorithm that outputs a decryption key, depending on both an input non-zero value $v \in \mathcal{R}_t$ and an identifier α_i . This algorithm takes as input the secret key sk , the non-zero value v , and the

key k_{α_i} . We sample the error e_3 in $\mathcal{D}_{q,\sigma}$ and compute the key $dk_{\alpha_i,v} = (k_1, \dots, k_m)$, where $k_i \leftarrow k_{\alpha_i} \cdot (sk_i - v)$.

Algorithm 3 KDer (sk_i, k_{α_i}, v)

Input: Takes as input the secret key sk_i , a key k_{α_i} , and a non-zero v the value that we want to see in the ciphertext.

Output: Outputs the derivation key $dk_{\alpha_i,v}$

- 1: Sample $e_3 \leftarrow \mathcal{D}_{q,\sigma}$
 - 2: **for** $j \in [m]$ **do**
 - 3: $k_j \leftarrow k_{\alpha_i} \cdot (sk_i - v) + t \cdot e_3$
 - 4: Set $dk_{\alpha_i,v} = (k_1, \dots, k_m)$
 - 5: **return** $dk_{\alpha_i,v}$
-

Decryption. The decryption is done using the functional decryption key $dk_{\alpha,v}$, without the knowledge of the secret key sk . This specificity guarantees partial decryption of the ciphertexts $\mathbf{c}_x = (c_{x_1}, \dots, c_{x_m})$ for value v and identifier α . **Meaning that for** $i \in [1, \dots, n]$, we test if $x_i == v$. **Leading us to** $f_v(x_1, \dots, x_n) = (1[x_1 == v], \dots, 1[x_n == v])$

Algorithm 4 Dec ($dk_{\alpha,v}, \mathbf{c}_x$)

Input: Take as input the derivation key $dk_{\alpha,v}$, and a vector of m ciphertexts $\mathbf{c}_x$

Output: Outputs the partially decrypted $\mathbf{x}$ noted $\mathbf{y} = (y_1, \dots, y_m)$.

- 1: **for** $i \in [m]$ **do**
 - 2: Parse $(c_1, c_2) \leftarrow c_{x_i}$ and $k_i \leftarrow dk_{\alpha,v}[i]$
 - 3: Compute $y_i^* \leftarrow c_1 + dk_i + v \cdot c_2$
 - 4: Reduce y_i^* into $[y_i^*]_t \in [0, t)$
 - 5: Reduce $[y_i^*]_t$ into $y_i \in (-q/2, q/2]$
 - 6: **return** $\mathbf{y} = (y_1, \dots, y_m)$ **where if** $y_i = 0 \rightarrow$ **output 1, else** $\rightarrow$ **output 0**
-

Correctness: Let $\lambda \in \mathbb{N}$ be an arbitrary security parameter, $(sk, pk) \leftarrow \text{Setup}(1^\lambda, pp)$ the secret and public keys, $\alpha \in \mathcal{R}_2$ an arbitrary identifier and $dk_{\alpha,v} \leftarrow \text{KDer}(sk, k_{\alpha}, v)$. For any ciphertext $c_{x_i} \leftarrow \text{Enc}(pk_i, c_i, \alpha)$ of a message $x_i \in \mathcal{M}$, it follows that:

$$y_i^* = c_1 + dk_i + v \cdot c_2 = (x_i - v) \cdot u + t \cdot E$$

where $E = e_{0,i} \cdot \alpha + e_1 + e_{\alpha} \cdot (sk_i - v) + e_3 + v \cdot e_2$.

To guarantee that the decryption works correctly, we need to choose the parameters so that the noise overhead does not exceed the range $(-q/2, q/2]$, that is:

$$\|y_i^*\| < q/2$$

Parameters. We choose the parameters according to the article of Mono *et al.* [23], with $\sqrt{n} V_{y_i^*} < q/2$, where $V_{y_i^*}$ is the variance of y_i^* . First, remark that $V_{y_i^*} = V_{tE}$, as we only need to check the correctness in the case decryption works, that is $u(x_i - v) = 0$. It

follows that:

$$\begin{aligned} V_{y_i^*} &= V_{tE} = V_{t(e_{0,i} \cdot \alpha + e_1 + e_{\alpha} \cdot (sk_i - v) + e_3 + v \cdot e_2)} \\ &= t^2 V_{e_{0,i} \cdot \alpha + e_1 + e_{\alpha} \cdot (sk_i - v) + e_3 + v \cdot e_2} \\ &= t^2 \sigma^2 (4n/3 + 2 + t^2/6) < q/2 \end{aligned}$$

where q is the ciphertext modulus, t the plaintext modulus, σ the derivation of the gaussian $\mathcal{D}_{q,\sigma}$ and n the degree of the quotient ring $\mathcal{R}$.

4.2 Updatable Functional Encryption

We now present the algorithms designed to achieve ciphertext updatability, as defined in Definition 3.3. In our model, we assume that token generation Algorithm 5 is run by a trusted authority **KC**. It takes as input the secret key and two labels: the original label δ_{in} , and the label δ_{out} to which we want to make the update. It outputs a token $tk_{\delta_{in} \rightarrow \delta_{out}}$.

Algorithm 5 TokGen ($sk, \delta_{in}, \delta_{out}$)

Input: Takes as input the secret key sk and labels $\delta_{in}, \delta_{out}$

Outputs: Outputs the update token $tk_{\delta_{in} \rightarrow \delta_{out}}$

- 1: Sample $\mathbf{e}_{tk} \leftarrow \mathcal{D}_{q,\sigma}^2$
 - 2: Compute $tk_{\delta_{in} \rightarrow \delta_{out}} \leftarrow pk \cdot (\delta_{out} - \delta_{in}) + t \cdot \mathbf{e}_{tk}$
 - 3: Output $tk_{\delta_{in} \rightarrow \delta_{out}}$
-

This update token can be used on a ciphertext to update its label using Algorithm 6.

Algorithm 6 Upd ($tk_{\delta_{in} \rightarrow \delta_{out}}, c_{\delta_{in}}$)

Input: Takes as input a token $tk_{\delta_{in} \rightarrow \delta_{out}}$ and a ciphertext $c_{\delta_{in}}$ encrypted under δ_{in} .

Outputs: Outputs $c_{\delta_{out}}$ encrypted under the label δ_{out}

- 1: Compute $c_{\delta_{out}} \leftarrow tk_{\delta_{in} \rightarrow \delta_{out}} + c_{\delta_{in}}$
-

Correctness: Let $\lambda \in \mathbb{N}$ be an arbitrary security parameter. Generate $(sk, pk) \leftarrow \text{Setup}(\lambda, n)$ and choose $\delta_{in}, \delta_{out}$ two labels. Let $x \in \mathcal{M}$ be an arbitrary message, $c_{\delta_{in}} \leftarrow \text{Enc}(pk, x, \delta_{in})$ and $dk_{f,\delta_{in}}, dk_{f,\delta_{out}}$ for labels $\delta_{in}, \delta_{out}$ respectively. We show that:

$$\text{Dec}(dk_{f,\delta_{out}}, c_{\delta_{out}}) = \text{Dec}(dk_{f,\delta_{in}}, c_{\delta_{in}})$$

with overwhelming probability. Since we assume that $dk_{f,\delta_{out}}$ is a valid decryption key, it is sufficient to show that $c_{\delta_{out}}$ is an encryption of x under the label δ_{out} . For every $i \in [n]$, we have:

$$\begin{aligned} c_{\delta_{out}} &= tk_{\delta_{in} \rightarrow \delta_{out}} + c_{\delta_{in}} \\ &= (b \cdot \delta_{out} + (e_1 + e_{tk,1}) + u \cdot x, a \cdot \delta_{out} + (e_2 + e_{tk,2}) + u) \\ &\in \text{Im}(\text{Enc}(pk, x, \delta_{out})) \end{aligned}$$

if and only if $(e_1 + e_{tk,1})$ and $(e_2 + e_{tk,2})$ are valid errors, which is the case in all RLWE encryption schemes.

5 Application Scenario and Protocol

We now describe a protocol to demonstrate the applicability of the construction. In this scenario, we consider a group of data owners $\mathbf{DO} = (d_1, \dots, d_m)$. We assume the existence of a trusted authority, or key curator $\mathbf{KC}$, that generates the master and functional decryption keys. The users $\mathbf{U} = (u_1, \dots, u_\ell)$, who make use of the data, are external entities only interacting with $\mathbf{KC}$ to request the decryption keys.

5.1 System and Threat model

To describe our protocol, we formally define the capabilities of the three entities involved and the threat model that covers the level of trust in each entity.

System Model. The considered system model consists of the following four entities:

- **Key Curator (KC):** A trusted authority responsible for the generation of the keys. It initializes the system and generates the secret key, while providing the public key to registered data owners $\mathbf{DO}$. Upon a user's request u_k for a value v and identifier α_j , it provides a functional derivation key $dk_{j,v}$.
- **Data Owners (DO = $(d_1, \dots, d_m)$):** Each d_j owns a private identifier α_j and registers in the system by sending α_j to $\mathbf{KC}$. The set $\mathbf{DO}$ is not bounded and a new owner of data d_{m+1} can dynamically register by sending their private identifier α_{m+1} . A data owner d_j is responsible for the encryption of their own data, using α_j and the public key.
- **Users ($\mathbf{U} = (u_1, \dots, u_\ell)$):** Users wish to learn the positions of a value v in a message $\mathbf{x}$. To do so, a user u_k sends a request to $\mathbf{KC}$ for the corresponding ciphertext c_x and obtains a derivation key $dk_{j,v}$.

Threat Model. In the considered threat model, $\mathbf{KC}$ is a trusted authority. As it owns the master secret key, it can generate an unbounded polynomial number of functional decryption keys and therefore has the ability to decrypt all the ciphertexts provided by $\mathbf{DO}$. On the other side, we assume that it provides the right functional decryption key $dk_{j,v}$ corresponding to $\mathbf{U}$'s requests. The owners of the data are not required to trust each other. We make the natural assumption that they encrypt the correct plaintexts. Specifically, the security of a data owner d_j 's data is guaranteed even if all other data owners are malicious. Users $\mathbf{U}$ are trusted only with the information they can recover from the ciphertexts and the decryption keys provided by $\mathbf{KC}$. Finally, we assume that users do not collude, which is a standard assumption in access-control systems where each user possesses individual credentials.

Setup. As a first step, $\mathbf{KC}$ runs the Setup algorithm to generate the keys (sk, pk) for the cryptosystem, defined in Section 4.1. On the other side, each data owner $d_j \in \mathbf{DO}$ generates their private identifier α_j . In the protocol overview ??, they register during the setup phase. However, registration in the system can be made dynamically at any given time. Eventually, d_j identifies itself by sending a message $\mathbf{m}_j$ to $\mathbf{KC}$, which answers with a message $\mathbf{m}_1$ containing the public key. Formally, $\mathbf{m}_j$ and $\mathbf{m}_1$ develop as follows: $\mathbf{m}_j = \langle t_j, c_{\alpha_j}, \sigma_{d_j}(\mathcal{H}(t_j \parallel c_{\alpha_j})) \rangle$ and $\mathbf{m}_1 = \langle t_1, pk, \sigma_C(\mathcal{H}(t_1 \parallel pk)) \rangle$ where t_j, t_1 are timestamps, σ_{d_j}, σ_C are d_j 's and $\mathbf{KC}$'s signatures, respectively, and c_{α_j} is the PKE encryption of $j \parallel \alpha_j$ for $\mathbf{KC}$'s public

key. Upon reception, each party verifies the freshness and integrity of the messages by respectively checking the timestamp and the signature. $\mathbf{KC}$ stores (j, α_j) for each registered data owner d_j . **Note:** In Section 4.1, the identifier α is generate in Algorithm 1. However, in a real-life scenario, data owners will generate their α and will then send their requests for access and keys to the $\mathbf{KC}$.

User Registration. On completion of $\mathbf{KC}$'s setup, a user $u_k \in \mathbf{U}$ sends a registration message reg to the key curator $\mathbf{KC}$ containing a set of one or more pairs (j, v) . Each pair corresponds to a decryption key request for the partial decryption of data collected by d_j , for a value v . The message is formed as: $\mathbf{m}_2 = \langle t_2, \text{reg}, \sigma_{u_k}(\mathcal{H}(t_2 \parallel \text{reg})) \rangle$.

Upon reception, $\mathbf{KC}$ verifies the freshness and integrity of the message by respectively checking the timestamp and the signature. $\mathbf{KC}$ first computes the functional decryption key $dk_{j,v} \leftarrow \text{KDer}(sk, v, \alpha_j)$ for each requested pair (j, v) . Then, it encrypts $\{dk_{j,v}\}$ into a ciphertext we denote c_{dk} , using u_k 's public key for PKE. It forwards it to u_k via the following message: $\mathbf{m}_3 = \langle t_3, c_{dk}, \sigma_C(\mathcal{H}(t_3 \parallel c_{dk})) \rangle$.

Again, u_k verifies the freshness and integrity of $\mathbf{m}_3$. If the message is valid, u_k stores the decryption keys $\{dk_{j,v}\}$ for further use.

Encryption. We assume d_j owns data $\mathbf{x}$ as plaintexts. Using the public key pk obtained from $\mathbf{m}_1$, d_j encrypts $\mathbf{x}$ into an RLWE ciphertext $c_{j,x} \leftarrow \text{Enc}(pk, \mathbf{x}, \alpha_j)$. He then outsources the data to the key curator as a message $\mathbf{m}_4 = \langle t_4, c_{j,x}, \sigma_{d_j}(\mathcal{H}(t_4 \parallel c_{j,x})) \rangle$.

Upon reception, $\mathbf{KC}$ verifies the freshness and integrity of the messages by respectively checking the timestamp and signature. If the verification succeeds, $\mathbf{KC}$ stores the ciphertexts obtained.

Data Access. Next, u_k sends a request for the encrypted data to $\mathbf{KC}$ via: $\mathbf{m}_5 = \langle t_5, \text{req}, \sigma_{u_k}(\mathcal{H}(t_5 \parallel \text{req})) \rangle$, where req denotes a request for a set of encrypted data, for example, data collected by a specific owner d_j or data collected during a specific time frame. Upon reception, $\mathbf{KC}$ verifies the message's integrity and freshness, and returns $\{c_{j,x}\}$ the corresponding set of such messages $c_{j,x}$ with the following message: $\mathbf{m}_6 = \langle t_6, \{c_{j,x}\}, \sigma_C(\mathcal{H}(t_6 \parallel \{c_{j,x}\})) \rangle$.

Upon reception, u_k verifies the freshness and integrity of the message. She can then run the functional decryption algorithm as described in Algorithm 4 using $dk_{j,v}$ to recover the partial decryption of a ciphertext $c_{j,x}$ with respect to the value v . A complete overview of this protocol is illustrated in ??.

Ciphertext Updatability. At any moment during the protocol, $\mathbf{KC}$ has the capability of generating an update token tk to update a ciphertext $c_{\delta,x}$ into a ciphertext $c_{\delta',x}$ for a new label δ' . This feature is only possible because we assumed that $\mathbf{KC}$ is a trusted authority. Otherwise, a malicious adversary that corrupts $\mathbf{KC}$ and at least one data owner could update any ciphertext into a newly formed ciphertext for this corrupted user. In that case, they can access any data provided by any user. In this protocol, we assume that ciphertext updatability is inherent to the Access step, as it involves updating data upon request of a user. It requires no additional security assumption, as $\mathbf{KC}$ already has access to all data.

6 Security Analysis

In this section, we formally prove that both our construction and use-case protocol are secure against an active adversary. We first provide in Section 6.1 a semantic security analysis to prove the security of the underlying MCFE scheme on which the AoS protocol is built. The security of the latter is covered in Section 6.2, where we address its *robustness* and *secure access* security, two notions detailed below.

6.1 Semantic Security of AoS

Definition 6.1 (IND-FE-CPA). Let $FE = (\text{Setup}, \text{Enc}, \text{KDer}, \text{Dec})$ be a functional encryption scheme for message space $\mathcal{M}$ and functionality space $\mathcal{F}$. We say that FE is indistinguishable under chosen plaintext attack, or IND-FE-CPA, if for all PPT adversary $\mathcal{A}$, it holds that the advantage of $\mathcal{A}$:

$$\text{Adv}_{FE, \mathcal{A}}^{\text{ind-fe-cpa}}(\lambda) = \left| \mathbb{P} \left[\text{Exp}_{FE, \mathcal{A}}^{\text{ind-fe-cpa-0}}(\lambda) = 1 \right] - \mathbb{P} \left[\text{Exp}_{FE, \mathcal{A}}^{\text{ind-fe-cpa-1}}(\lambda) = 1 \right] \right|$$

is negligible in λ , where the experiment is defined as follows.

$\text{Exp}_{FE, \mathcal{A}}^{\text{ind-fe-cpa-}\beta}(\lambda)$	
<u>Setup(1^λ)</u>	<u>Challenge(x_0, x_1)</u>
$L, V \leftarrow \emptyset$	$L \leftarrow L \cup \{(x_0, x_1)\}$
$(\text{sk}, \text{pk}) \leftarrow \text{Setup}(1^\lambda)$	$c_\beta \leftarrow \text{Enc}(\text{pk}, x_\beta, \alpha)$
$\alpha \leftarrow \mathcal{R}_2$	<u>Finalize(β')</u>
<u>KeyDer(v)</u>	If $\exists v \in V, (x_0, x_1) \in L;$
$V \leftarrow V \cup \{v\}$	$f_v(x_0) \neq f_v(x_1) :$
$\text{dk}_v \leftarrow \text{KDer}(\text{sk}, v, \alpha)$	return $\perp$
return dk_v	return $\beta = \beta'$

Figure 1: Selective IND-FE-CPA security experiment parametrized by a bit $\beta \in \{0, 1\}$, a PPT adversary $\mathcal{A}$ and a security parameter λ .

The restriction enforced in **Finalize** is necessary to prevent the adversary from trivially winning the game. Concretely, the adversary is only granted decryption keys through the **KeyDer** oracle, and each query for a value v is recorded in V . If the adversary has previously obtained a key for v , it cannot later submit a challenge pair (x_0, x_1) with $x_0 = v$ and $x_1 \neq v$ (or vice versa): decrypting the challenge ciphertext with that key would immediately reveal β , since $f_v(x_0) \neq f_v(x_1)$ in this case. The **Finalize** check formalizes this by aborting the experiment whenever such a trivial distinguisher exists. We note that the two plaintexts submitted in a single challenge query must differ (a fresh challenge is required each time), and the total number of **KeyDer** and **Challenge** queries is bounded polynomially in λ , as required of any PPT adversary.

To prove the IND-FE-CPA security of AoS, we implement a reduction to the security of the traditional RLWE from Definition 3.6.

THEOREM 6.2. *The MCFE scheme AoS is IND-FE-CPA secure under the RLWE hardness assumption.*

PROOF. To prove Theorem 6.2, we proceed by a game reduction from the AoS IND-FE-CPA game to the IND-CPA game for Definition 3.6. We introduce two PPT algorithms: $\mathcal{A}$ is the adversary in the RLWE game for challenger C , and $\mathcal{B}$ is the adversary in the AoS game in which $\mathcal{A}$ is the challenger. We show that, if $\mathcal{B}$ has a non-negligible advantage in the AoS game, then $\mathcal{A}$ has a non-negligible advantage in the RLWE game.

Setup of RLWE game: The challenger C generates the secret and public key pair (sk, pk) for the RLWE game and forwards pk to $\mathcal{A}$. The latter submits two plaintexts x_0, x_1 to C , who picks a bit $\beta \in \{0, 1\}$ at random and computes $c_\beta \leftarrow \text{Enc}(\text{pk}, x_\beta)$, as:

$$c_\beta = \begin{cases} c_1 = b \cdot u + e_1 + x_\beta \\ c_2 = a \cdot u + e_2 \end{cases}$$

where $u \leftarrow \mathcal{R}_2$ is a ternary polynomial, and $e_1, e_2 \leftarrow \mathcal{D}_{q, \sigma}$ are sampled from a discrete Gaussian. The challenger C returns c_β to $\mathcal{A}$.

Setup of AoS game: Then, $\mathcal{A}$ samples $u' \leftarrow \mathcal{R}_2$ and alters the ciphertext $c_\beta = (c_1, c_2)$ into $c'_\beta = (c'_1, c'_2)$ formed as follows: $c'_1 \leftarrow c_1 \cdot u'$ and $c'_2 \leftarrow c_2 \cdot u' - u'$.

$$c'_\beta = \begin{cases} c'_1 = b \cdot u \cdot u' + e_1 \cdot u' + x_\beta \cdot u' \\ c'_2 = a \cdot u \cdot u' + e_2 \cdot u' - u' \end{cases}$$

This alteration is valid, as it solely relies on the challenge ciphertext c_β and the polynomial u' generated by $\mathcal{A}$, and is hence independent from any private parameter held by the challenger C .

Challenge: The adversary $\mathcal{A}$ forwards the altered ciphertext c'_β to the adversary $\mathcal{B}$ for the AoS game. Recall that this adversary is assumed to have a non-negligible advantage in this game. Therefore, the goal of $\mathcal{A}$ is to replicate this game for bit β and challenge plaintexts x_0, x_1 , to have $\mathcal{B}$ acting as an oracle in the RLWE game against C .

We remark that the ciphertext c'_β formed by $\mathcal{A}$ has the structure of a ciphertext for the MCFE scheme AoS, if $\mathcal{A}$ sets $\alpha \leftarrow (u \cdot u')$; $e'_1 \leftarrow (e_1 \cdot u')$; $e'_2 \leftarrow (e_2 \cdot u')$. Formally, c'_β is defined as:

$$c'_\beta = \begin{cases} b \cdot \alpha + e'_1 + u' \cdot x_\beta \\ a \cdot \alpha + e'_2 + u' \end{cases}$$

$\mathcal{A}$ eventually sends $\text{pk}, x_0, x_1, c'_\beta$ to $\mathcal{B}$. We note β' the guess made by $\mathcal{B}$ on the bit β chosen by C . Now, $\mathcal{B}$ forwards β' to $\mathcal{A}$, which submits this same guess β' to C in the RLWE game for which it is the adversary.

If we denote ϵ the advantage of $\mathcal{B}$ in the AoS game, $\mathcal{A}$ has at least an advantage ϵ in breaking the RLWE game against C . It follows that, if $\mathcal{B}$ breaks AoS (i.e. it is not IND-FE-CPA secure), then $\mathcal{A}$ also breaks the RLWE IND-CPA game against C . However, RLWE is IND-CPA secure by assumption, which concludes the proof. $\square$

6.2 Security of the protocol

This section addresses the security of the AoS protocol described in Section 5 or, more accurately, proves that our protocol satisfies *robustness* and *secure access*.

Robustness guarantees that any party can access data they are authorized to retrieve. In its formal security Definition 6.3, an active PPT adversary $\mathcal{A}$ can interfere with the communications $\mathbf{m}_j$ and $\mathbf{m}_1, \dots, \mathbf{m}_6$, but cannot influence the execution of the protocol. In this context, allowing $\mathcal{A}$ to corrupt users is meaningless, as they cannot influence the execution of **KC** and **DO**. However, it can impersonate **KC** or a data owner $d \in \mathbf{D}$ to send inappropriate initial messages to the parties and upload invalid contents. In the presence of such active adversary, we expect the protocol to guarantee that *all* authorized users can *correctly* retrieve partial decryption of the data.

Definition 6.3 (Robustness). A protocol Π for data owners **DO** = $(d_1, \dots, d_m)$, key curator **KC** and users $\mathbf{U} = (u_1, \dots, u_\ell)$ is robust, if for all PPT adversary $\mathcal{A}$, it holds that:

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{robust}}(\lambda) = \left| \mathbb{P} \left[\text{Exp}_{\Pi, \mathcal{A}}^{\text{robust}}(\lambda) \rightarrow \text{true} \right] - \frac{1}{2} \right|$$

is negligible in λ , where the experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{robust}}$ is defined as follows.

```

ExpΠ, Arobust(λ)
  L ← ∅ ; forged ← false
  (pp, stKC, stD, stU) ← Setup(1λ)
  (u*, req*) ← A(1λ, pp, {msgd}d∈D : Orobust)
  Parse req* as (idx, event)
  If (forged = true) ∨
    [∃ x* such that (x*, α*, c*) ∈ L
     ∧ c* is encrypted under α*
     ∧ x* ≠ (res ← (stu*, c*)) ] :
    return true

```

where O_{robust} are the oracles in the robustness game, formally defined in Figure 2.

This security property is defined by an experiment involving a challenger $\mathcal{C}$, acting as **KC**, and a PPT adversary $\mathcal{A}$ interacting with the protocol Π . $\mathcal{C}$ first runs **Setup**, generating initial states for all parties and providing each data owner $d_j \in \mathbf{D}$ with an initialization message msg_{d_j} . Upon receiving msg_{d_j} , d_j verifies that msg_{d_j} is well formed and updates its local state. All initialization messages are revealed to $\mathcal{A}$, which can invoke the oracle **Init_D**(d_j, msg_{d_j}) to further initialize owners, receiving a boolean response indicating success. User registration proceeds similarly: $\mathcal{A}$ may call **UserReg**($u_k, \{v, \alpha\}$) to register a user u_k with request $\{v, \alpha\}$ and, upon success, an update message is returned. The user then updates its local state by calling **Init_U** after verifying this update message.

The adversary can request a data owner d_j to upload a data x with identifier α by querying **DataUpload**. The oracle runs **Enc** to produce a ciphertext c , and executes **Upload** with d_j 's local state and (x, c) , to output the upload message. A list L keeps record of every entry x, α, c . The upload message is revealed to $\mathcal{A}$.

The adversary can also invoke **Upload**(d_j, msg) to upload arbitrary information. If msg is valid and no entry for x exists in L , the oracle returns x and sets a flag $\text{forged} = \text{true}$, indicating successful forgery.

Oracles O_{robust}

Init_D (d_j, msg_{d_j})	DataUpload (d_j, x, α_j)
If $d_j \notin \mathbf{D}$: return $\perp$	If $d_j \notin \mathbf{D}$: return $\perp$
If verification fails:	$c \leftarrow \text{Enc}(\text{st}_{d_j}, x, \alpha_j)$
return $\perp$	$\text{msg} \leftarrow \text{Upload}(\text{st}_{d_j}, x, c)$
Else: update st_{d_j} with msg_{d_j}	$L \leftarrow L \cup \{x, \alpha_j, c\}$
return true	return (x, msg)
Init_U (u_k, msg_{u_k})	Upload (d_j, msg)
If $u_k \notin \mathbf{U}$: return $\perp$	If $d_j \notin \mathbf{D}$: return $\perp$
If verification fails:	Extract (x, c) from msg
return $\perp$	If $\exists (x, *, *) \in L$: return $\perp$
Else: update st_{u_k} with msg_{u_k}	If Init_D (d_j, msg_{d_j}) = true:
return true	$L \leftarrow L \cup \{x, *, c\}$
	$\text{forged} = \text{true}$
UserReg (reg)	return x
Parse reg as $(u_k, \{v, \alpha\})$	return $\perp$
If $u_k \notin \mathbf{U}$: return $\perp$	DataAccess (req)
$\text{st}_{u_k} \leftarrow \text{KDer}(u_k, \{v, \alpha\})$	Parse req as $(\text{id}_x, \text{event})$
return msg_{u_k}	If $\exists (x, *, c) \in L$: return c
	Else: return $\perp$

Figure 2: Oracles for the robustness game.

To access encrypted data, $\mathcal{A}$ queries the **DataAccess** oracle with a data access token req . The token is structured as a specific identifier id_x along with a potential event name event. Eventually, $\mathcal{A}$ receives the encrypted data c from the records in L .

There are two situations in which the adversary wins:

- (1) A data owner uploads x under α_j , but no authorized user (with decryption keys for α_j) can correctly recover x .
- (2) A user gains access to data that was never uploaded by any data owner (i.e., the data is forged).

In either case, if there exists a user u^* and data identifier id_x^* such that u^* fails to decrypt or the forged flag is set, $\mathcal{A}$ wins the game.

Secure Access: The second security definition we introduce ensures that unauthorized users cannot obtain any partial content of ciphertexts. This property is defined by a security game similar to the robustness game, but now the adversary $\mathcal{A}$ attempts to guess a random bit β chosen at the beginning of the game. The game maintains three lists: L for encrypted data, Ch for the challenges, and Cr for corrupted users. Unlike the robustness game, $\mathcal{A}$ may call a corruption oracle **Corrupt** to compromise users. Thus, **UserReg** is modified to permit corruption before and after user registration. However, $\mathcal{A}$ cannot run **Init_U**, since only $\mathcal{C}$ can initialize users. Honest users do not write data, so they cannot aid the adversary in leaking data. Similarly, the adversary has no capability to impersonate data owners.

The adversary issues a challenge via **Chall** by specifying a data owner d_j , two plaintexts x_0 and x_1 , and α . The challenger encrypts and stores x_β in L . To prevent trivial wins, every corrupted user's registration requests $\{v, \alpha'\}$ must be such that $\alpha' \neq \alpha$. The game ends when $\mathcal{A}$ outputs a guess β' , and the adversary wins if $\beta' = \beta$.

Definition 6.4 (Secure Access). A protocol Π for data owners $\mathbf{DO} = (d_1, \dots, d_m)$, key curator $\mathbf{KC}$ and users $\mathbf{U} = (u_1, \dots, u_t)$ is secure with respect to secure access, if for all PPT adversary $\mathcal{A}$, it holds that

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{s-acc}}(\lambda) = \left| \mathbb{P} \left[\text{Exp}_{\Pi, \mathcal{A}}^{\text{s-acc}}(\lambda) \rightarrow \text{true} \right] - \frac{1}{2} \right|$$

is negligible in λ , where the experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{s-acc}}$ is defined as follows. where $\mathcal{O}_{\text{s-acc}}$ are the oracles in the secure-access game, formally defined in Figure 3.

$\text{Exp}_{\Pi, \mathcal{A}}^{\text{s-acc}}(\lambda)$
 $L, \text{Ch}, \text{Cr} \leftarrow \emptyset; \beta \leftarrow \{0, 1\}$
 $(\text{pp}, \text{st}_{\text{KC}}, \text{st}_{\text{D}}, \text{st}_{\text{U}}) \leftarrow \text{Setup}(1^\lambda)$
 $\beta' \leftarrow \mathcal{A}(1^\lambda, \text{pp}, \{\text{msg}_d\}_{d \in \mathbf{D}}; \mathcal{O}_{\text{s-acc}})$
 Return $(\beta = \beta')$

Oracles $\mathcal{O}_{\text{s-acc}}$

InitD (d_j, msg_{d_j})	DataAccess (req)
If $d_j \notin \mathbf{D}$: return $\perp$	Parse req as $(\text{id}_x, \text{event})$
If verification fails:	If $\exists (x, \alpha, c) \in L$:
return $\perp$	return c
Else: update st_{d_j} with msg_{d_j}	Corrupt (u_k)
return true	If $u_k \notin \mathbf{U}$: return $\perp$
UserReg (reg)	For all $\alpha \in \text{Ch}$:
Parse reg as $(u_k, \{v, \alpha'\})$	If $\alpha \in \text{reg}_{u_k}$: return $\perp$
If $u_k \notin \mathbf{U}$: return $\perp$	$\text{Cr} \leftarrow \text{Cr} \cup \{u_k\}$
If $(u_k, \alpha') \in \text{Cr}$:	return st_{u_k}
For all $\alpha \in \text{Ch}$:	Chall (d, x_0, x_1, α)
If $\alpha = \alpha'$: return $\perp$	If $d \notin \mathbf{D}$: return $\perp$
$\text{st}_{u_k} \leftarrow \text{KDer}(u_k, \{v, \alpha'\})$	For all $u_k \in \text{Cr}$:
If $u_k \in \text{Cr}$: return st_{u_k}	If $\alpha \in \text{reg}_{u_k}$: return $\perp$
Else return u_k	$c \leftarrow \text{Enc}(\text{st}_d, x_\beta, \alpha)$
DataUpload (d_j, x, α_j)	$L \leftarrow L \cup \{(x_\beta, \alpha, c)\}$
If $d_j \notin \mathbf{D}$: return $\perp$	$\text{Ch} \leftarrow \text{Ch} \cup \{\alpha\}$
$(x, c) \leftarrow \text{Enc}(\text{st}_{d_j}, x_\beta, \alpha_j)$	return c
$L \leftarrow L \cup \{x, \alpha_j, c\}$	
return x	

Figure 3: Oracles for the secure access game.

THEOREM 6.5. If the signature scheme $\mathcal{S}$ is EUF-CMA secure, the construction AoS is robust in the random oracle model.

PROOF. We prove the theorem through a sequence of games involving a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ that tries to break the robustness game. The experiments are described in detail below.

Game 0: The initial game is the original robustness game, and we denote $\Pr[\text{Game}_{i, \mathcal{A}}]$ the probability that $\mathcal{A}$ wins in game i .

Game 1: This game proceeds the same as the initial game, except that whenever $\mathcal{A}$ queries the oracle Init_U with an initial message msg_{u_k} for a user $u_k \in \mathbf{U}$ that includes a valid signature not previously generated by $\mathcal{C}$ in response for some query to the oracle UserReg , the game aborts. The validity of the signature can be verified by running the initialization algorithm Init_U with the public parameter pp and the user's initial local state st_{u_k} .

Claim 1. $|\Pr[\text{Game}_{0, \mathcal{A}}] - \Pr[\text{Game}_{1, \mathcal{A}}]| \leq \epsilon_0$, where ϵ_0 is the advantage of $\mathcal{A}$ to break the EUF-CMA security of the signature scheme $\mathcal{S}$.

We construct an adversary $\mathcal{B}$ that breaks the game that defines EUF-CMA security, given an adversary $\mathcal{A}$ who can successfully forge a valid initial message with a signature not generated by $\mathcal{C}$ for some user in the system. $\mathcal{B}$ can easily simulate the robustness game for $\mathcal{A}$, except that it does not run the key generation of the signature scheme $\mathcal{S}$. Therefore, it needs to call the signing oracle to obtain signatures for the initial messages generated in response to queries to the simulated oracle UserReg . Instead, upon receiving the registration token from $\mathcal{A}$, $\mathcal{B}$ queries the signing oracle in the EUF-CMA game and attaches the obtained signature to the initial message. When $\mathcal{A}$ queries Init_U with a valid initial message msg_u for some user u , $\mathcal{B}$ takes the timestamp and the encrypted key $c_{dk_{\alpha, o}}$, along with the corresponding signature, and forwards it as its output in the EUF-CMA game. Since the message was not signed by the challenger in $\mathcal{B}$'s game, $\mathcal{B}$ wins the game whenever $\mathcal{A}$ wins the simulated game, with the same probability.

Game 2: This game proceeds as the previous one, with the exception that $\mathcal{C}$ aborts the game when $\mathcal{A}$ queries the Init_D oracle with a valid initial message msg_d for a data owner d , which is not generated during the setup process. The validity of the signature can be verified by running the initialization algorithm Init_D with the public parameter pp and the data owner's initial local state st_d .

Claim 2. $|\Pr[\text{Game}_{1, \mathcal{A}}] - \Pr[\text{Game}_{2, \mathcal{A}}]| \leq \epsilon_1$, where ϵ_1 is the advantage of $\mathcal{A}$ to break the EUF-CMA security of the signature scheme $\mathcal{S}$.

The proof of this claim is similar to the previous one. The adversary $\mathcal{B}$ for the EUF-CMA game simulates the robustness game for the adversary $\mathcal{A}$. Since it does not possess the signing key of $\mathcal{C}$, $\mathcal{B}$ queries its signing oracle to obtain the signatures for the initial messages for each $d \in \mathbf{D}$ in the system during the setup process. When $\mathcal{A}$ queries Init_D with a valid initial message for some $d \in \mathbf{D}$, $\mathcal{B}$ takes the timestamp and ciphertext (t, c) , and the corresponding signature as its output. To win the game, $\mathcal{A}$ has to generate a valid signature for a message that is not previously signed by $\mathcal{B}$. Therefore, $\mathcal{B}$ wins the game when $\mathcal{A}$ wins the simulated game. It follows that $|\Pr[\text{Game}_{1, \mathcal{A}}] - \Pr[\text{Game}_{2, \mathcal{A}}]| \leq \epsilon_1$. The adversary can win the final game if either of the two winning conditions is satisfied: (i) it manages to generate an invalid data entry which was not previously generated by any $d \in \mathbf{D}$ and successfully uploads it to $\mathbf{KC}$ by calling the DataUpload ; (ii) The specified user cannot correctly retrieve some data content genuinely uploaded by a data owner.

After the consecutive modifications in the previous game, $\mathcal{A}$ is not able to manipulate the local states of either $\mathbf{D}$ or $\mathbf{U}$, since the challenger will abort the game. The correctness of the underlying cryptographic primitives guarantees that the specified authorized user must be able to correctly retrieve the data content. Therefore, the only scenario where $\mathcal{A}$ can win the game is by calling DataUpload to upload invalid contents, resulting in decryption failure. Therefore, the adversary must be able to forge a valid signature for the valid content so that it can be successfully accepted by the $\mathbf{KC}$.

Claim 3. $\Pr[\text{Game}_{2, \mathcal{A}}] \leq \frac{1}{|\mathbf{D}|} \cdot \epsilon_2$, where ϵ_2 is the advantage of $\mathcal{A}$ to break the EUF-CMA security of the signature scheme $\mathcal{S}$.

Given an adversary $\mathcal{A}$ breaking the robustness of AoS, we show that an adversary $\mathcal{B}$ for the EUF-CMA security game can be constructed as follows. $\mathcal{B}$ first selects a random index j from the range $[[D]]$ before simulating the robustness game for $\mathcal{A}$. Then, it initializes the cryptographic primitives employed in the protocol, except for the signing key for the data owner d_j , but includes the verification key it obtains from the EUF-CMA game in the public parameters. After that, $\mathcal{B}$ runs a local copy of $\mathcal{A}$ and simulates to it the oracles $\mathcal{O}_{\text{robust}}$. During the simulation, whenever $\mathcal{B}$ needs to generate a signature on behalf of d_j , it queries the signing oracle in its game. When $\mathcal{A}$ queries **DataUpload** with valid content not already in L for $d \in D$, $\mathcal{B}$ aborts the game if $d \neq d_j$. Otherwise, it records the signature and stores the content in L . When $\mathcal{A}$ specifies a user with an access request req as its output, $\mathcal{B}$ retrieves the encrypted data with the timestamp (t, c) from L and outputs it with the corresponding signature, hence giving a perfect simulation. With the use of the signing oracle in the EUF-CMA game, $\mathcal{B}$ can answer all queries from $\mathcal{A}$ appropriately according to the protocol specification. Additionally, the valid forgery submitted by $\mathcal{A}$ is guaranteed not to have been previously generated by d_j . Therefore, when $\mathcal{A}$ wins the simulated game, $\mathcal{B}$ wins the EUF-CMA game.

From claims 1, 2 and 3, we conclude that

$$\Pr[\text{Game}_{0,\mathcal{A}}] \leq \epsilon_0 + \epsilon_1 + \frac{1}{|D|} \cdot \epsilon_2,$$

which proves the theorem. $\square$

THEOREM 6.6. *If PKE is IND-CPA secure, $\mathcal{S}$ is EUF-CMA secure, and the AoS scheme is IND-FE-CPA secure, the construction AoS is secure with respect to secure access in the random oracle model.*

PROOF. We now provide a proof of this theorem using a sequence of games, which we describe below.

Game 0: The first game is the experiment which defines secure access in Definition 6.4.

Game 1: This game proceeds as the previous one, except that when $\mathcal{A}$ requests to initialize d_j with a valid initial message msg_{d_j} , $\mathcal{C}$ aborts the game.

Claim 4. $|\Pr[\text{Game}_{0,\mathcal{A}}] - \Pr[\text{Game}_{1,\mathcal{A}}]| \leq \epsilon_0$, where ϵ_0 is the advantage of $\mathcal{A}$ to break the EUF-CMA security of the signature scheme $\mathcal{S}$.

Game 2: During this game, $\mathcal{C}$ maintains a list K indexed by users to record the decryption keys. Specifically, when $\mathcal{C}$ runs **KDer** to generate the decryption key $\text{dk}_{\alpha,v}$ in response to a user registration request (by querying the oracle **UserReg** with a registration token reg), it records $\text{dk}_{\alpha,v}$ in K . Meanwhile, $\mathcal{C}$ runs the encryption algorithm **PKE.Enc**, to encrypt a random element in the same set as $\text{dk}_{\alpha,v}$ using u_k 's public key pk_{u_k} . In the case that u_k is corrupted or $\mathcal{A}$ requests to corrupt u_k after registration, $\mathcal{C}$ retrieves the decryption key from K and provides it to the adversary.

Claim 5. $|\Pr[\text{Game}_{1,\mathcal{A}}] - \Pr[\text{Game}_{2,\mathcal{A}}]| \leq \epsilon_1$, where ϵ_1 is the advantage of $\mathcal{A}$ to break the IND-CPA security of the public key encryption scheme PKE.

We prove this claim by showing the construction of a PPT algorithm $\mathcal{B}$ that can distinguish between **Game1** and **Game2** given the public key and the oracle access in the IND-CPA security game

for the public key encryption scheme PKE. The main idea is that $\mathcal{B}$ submits its challenge for the IND-CPA game in its simulation of the secure access game for AoS, obtaining a hybrid between the above two games. Specifically, $\mathcal{B}$ randomly selects a user $u_k \in U$ and chooses a random bit β at the beginning of the simulated game. When generating the decryption key for u_k , $\mathcal{B}$ calls the challenge oracle in its game with a query $(\text{dk}_{\alpha,v}, r)$ to obtain a ciphertext c , where r is a random element. Then, it includes c in the message as a response to $\mathcal{A}$'s query to **UserReg** with respect to u_k . Whenever $\mathcal{A}$ requests to corrupt u_k to learn its local state, $\mathcal{D}$ terminates the simulation and outputs 0. When $\mathcal{A}$ outputs a guess β' for the random bit β , $\mathcal{B}$ outputs 1 if $\beta' = \beta$ and 0 otherwise. If the challenge oracle in the IND-CPA game always returns an encryption of $\text{dk}_{\alpha,v}$, the simulated game is identical to **Game1**. If it always returns an encryption of r , it corresponds to **Game2**. Thus, $\mathcal{B}$ effectively simulates a hybrid game of **Game1** and **Game2**. Regardless of the data specified by $\mathcal{A}$ as its challenge, there must be at least a user in the system who has access to it. In a nutshell, the assumption ensures that there exists at least one user who remains uncorrupted during the game and $\mathcal{B}$ has probability at least $\frac{1}{|U|}$ of selecting that user, which provides a lower bound on its success probability ($\frac{1}{|U|} \cdot \epsilon_1$). Finally, if $\mathcal{A}$ manages to win the game without taking over any user, $\mathcal{B}$'s probability is bounded by ϵ_1 . Thus we have

$$\frac{1}{|U|} \cdot \epsilon_1 \leq \Pr[\mathcal{B} \rightarrow \text{true}] \leq \epsilon_1,$$

We conclude by analyzing the advantage of $\mathcal{A}$ in **Game2**. It is clear that $\mathcal{A}$ cannot learn any part of a user's local state without corrupting it, as the local state is replaced with a random element. Furthermore, $\mathcal{A}$ cannot impersonate KC to have any owner encrypt and upload data contents using an inappropriate key crafted by the adversary. A successful adversary in this final game would need to gain some advantage in breaking the IND-FE-CPA security of AoS.

Claim 6. $\Pr[\text{Game}_{2,\mathcal{A}}] = \epsilon_2$, where ϵ_2 is the advantage of an efficient adversary that breaks the IND-FE-CPA security of the MCFE scheme AoS.

We construct an adversary $\mathcal{B}$ for the IND-FE-CPA game from an adversary $\mathcal{A}$ that breaks the secure access property of AoS. The main idea is that $\mathcal{B}$ simulates to $\mathcal{A}$ the **Game2** in such a way that $\mathcal{B}$'s advantage in winning its game is exactly the same as the advantage of $\mathcal{A}$ can gain in **Game2**. Upon receiving the public key pk from its challenger, $\mathcal{B}$ initializes AoS without generating the corresponding secret key. After that, $\mathcal{B}$ invokes the oracles that $\mathcal{A}$ has access to, except for **Chall**. When $\mathcal{A}$ queries **Chall** by specifying a data owner d_j , two plaintexts x_0, x_1 and α' as its challenge, $\mathcal{B}$ checks that if there is no corrupt user can get access the content of that entry according to α' , it calls **Chall** with (x_0, x_1, α') to obtain the ciphertext c and sends it to $\mathcal{A}$. At this point, as long as $\mathcal{B}$ does not corrupt users who can get access to the challenged data, the game for semantic security of AoS will never return an error, as $(\alpha = \alpha')$ will never hold for any $\alpha \in \text{Ch}$. Thus, $\mathcal{B}$ provides a perfect simulation. Moreover, the simulated game is fully determined by the random bit selected in $\mathcal{B}$'s game. Therefore, the probability that $\mathcal{B}$ correctly guesses the random bit is the same as the probability that $\mathcal{A}$ wins the simulated game, hence: $\Pr[\text{Game}_{2,\mathcal{A}}] = \epsilon_2$.

From claims 4,5 and 6, we have: $\Pr[\text{Game } 0] \leq \epsilon_0 + \epsilon_1 + \epsilon_2$.

In conclusion, assuming the public key encryption scheme is IND-CPA secure, the signature is EUF-CMA secure and MCFE scheme is IND-FE-CPA secure, AoS is secure with respect to secure access. $\square$

7 Evaluation

To evaluate the practicality of our scheme, we implemented a proof-of-concept of AoS using the *Go* programming language and *Lattigo* library [1]. For testing and benchmarking, we followed best practices by using the Golang standard test package. Our experiments were conducted in a single-threaded environment on a PC with an Intel Core i5-1235 CPU @ 4.4GHz and 16GB of memory. Table 2 summarizes our benchmarks in terms of running time and memory usage.

Open Science & Reproducible Research. To support open and reproducible science, we made our implementation source code available online¹ (anonymized version).

7.1 AoS Benchmarks

To benchmark AoS, we follow the notation and algorithms in Section 4.1, using a modular approach for implementing different functions of $AoS = \{\text{Setup, Enc, KDer, Dec, TokGen, Upd}\}$. As shown in Table 1, we define four parameter sets – **Params** = {S, M, L, XL} – to evaluate performance across different system scales. For example, the smallest set (S) uses input vectors m of size 100 with a polynomial ring degree of $\log_2 N = 12$ and ciphertext modulus $\log_2 q = 39$. Additionally, for all the parameter sets, we have used the same plaintext modulus t equal to 2^{16} .

Table 1: Parameters for Benchmarking

Params	m	$\log_2 N$	$\log_2 q$	$\log_2 t$
Small	100	12	39	16
Medium	200	13	42	16
Large	500	14	43	16
XLarge	1000	15	47	16

Table 2: Ace of SPADE Benchmarks

Component	Time (ms/op)	Memory Allocations (KB/op)
B1 with parameter set S		
Setup	22.4327	25998
Encryption	0.4933	353
KeyDerivation	4.8752	6439
Decryption	3.5918	6410
TokenGen	0.4691	193
Update	0.0244	64
B2 with parameter set M		
Setup	74.4276	103384
Encryption	1.0265	706
KeyDerivation	7.1626	25678
Decryption	9.3621	25619
TokenGen	0.9092	386
Update	0.0406	128
Component	Time (ms/op)	Memory Allocations (KB/op)
B3 with parameter set L		
Setup	372.7643	515183
Encryption	1.1618	1412
KeyDerivation	39.7671	128164
Decryption	46.1236	128047
TokenGen	1.1352	772
Update	0.0564	256
B4 with parameter set XL		
Setup	1485.6854	2058371
Encryption	3.6025	2824
KeyDerivation	123.7577	512327
Decryption	150.0868	512095
TokenGen	2.1088	1544
Update	0.1498	512

As shown in Table 2, the **running time** and **memory usage** of AoS increases with the change of parameters. The comparison between the smallest (B1) and largest benchmarks (B4), which involves increasing the number of samples by 10 $\times$, using a larger

¹<https://anonymous.4open.science/r/AoS-76C7/>

ring degree, and a larger ciphertext modulus, shows that the average running time is 25 $\times$ longer, and memory usage is 43 $\times$ higher.

8 Comparison with Other Work

Comparing AoS with existing constructions is challenging because closely related scheme often rely on fundamentally different cryptography assumptions and provide different functionality. As a result, direct performance or security comparisons may be misleading.

SPADE. The closest construction to AoS is SPADE [24] which introduced the notion of partial decryption for qualitative and quantitative analysis over encrypted data. However, SPADE is based on additive ElGamal and on cyclic group assumption where AoS relies on RLWE assumption and therefore provides post-quantum security guarantees. In addition, AoS extends the SPADE construction with ciphertext updatability. Consequently, while SPADE serves as the main inspiration for AoS, the two constructions target different security settings and offer different functionalities.

IPFE. RLWE-based Inner-Product Functional Encryption (IPFE) [21] schemes IPFE enables the computation of the inner-product between an encrypted vector and a function vector provided through a functional decryption key. In contrast, AoS is designed for qualitative analysis through selection partial decryption. AoS determines whether each encrypted element matches with a target value while revealing no additional information about the plaintext.

PKEET/PCE. Public-Key Encryption with Equality Test (PKEET) allows a third party to determine whether two ciphertexts encrypt the same plaintext, while Plaintext-Checkable Encryption (PCE) enables the verification of whether a ciphertext encrypts a guessed plaintext. They are both classical public-key encryption primitives and do not support functional decryption through FE-style derivation keys and they operate in a single-client setting and do not support the MCFE architecture used in AoS. PKEET and PCE constructions are primarily based on classical number-theoretic assumptions and do not provide post-quantum security. Finally, these schemes do not support ciphertext updatability or selective partial decryption over encrypted vectors.

To the best of our knowledge, AoS is currently the only RLWE-based and post-quantum secure FE-style construction supporting selective partial decryption for both quantitative and qualitative data while additionally integrating ciphertext updatability in a multi-client setting.

9 Conclusion

FE constructions still suffer from a lack of versatility, as the current state-of-the-art largely consists of constructions supporting addition, inner product, or quadratic polynomials. This limitation motivated the design of AoS, a new FE construction that shows that multi-client FE can be extended to a broader range of functionalities. More precisely, the proposed construction supports partial decryption in a plethora of different scenarios and allows qualitative analysis to be performed on encrypted data. Furthermore, the proposed scheme has two important features: (1) it is instantiated from the RLWE problem and it is proven to be quantum secure. This is a major step

forward compared to previous works, as it not only achieves better security, but also complies with PKE standards and most state-of-the-art constructions in FE; (2) Supports ciphertext updateability – an important feature that is currently absent from most existing FE constructions.

Future Directions. We believe that lattices were the best fit for our construction. However, this does not mean that partial decryption is limited to this particular hardness assumption. As already presented in [24], such construction can be designed on top of the ElGamal public-key encryption scheme. We believe that the *trapdoor* on which partial decryption relies can be extended to other families of PKE schemes, in particular to code-based PKE or relying on the NTRU problem. Eventually, we also believe that such construction can be done on top of any PKE scheme.

References

- [1] Lattigo v6. Online: <https://github.com/tuneinsight/lattigo> (Aug 2024), ePFL-LDS, Tune Insight SA
- [2] Abdalla, M., Bourse, F., De Caro, A., Pointcheval, D.: Simple functional encryption schemes for inner products. In: Katz, J. (ed.) *Public-Key Cryptography – PKC 2015*. pp. 733–751. Springer Berlin Heidelberg, Berlin, Heidelberg (2015)
- [3] Agrawal, S., Freeman, D.M., Vaikuntanathan, V.: Functional encryption for inner product predicates from learning with errors. In: *Advances in Cryptology – ASIACRYPT 2011: 17th International Conference on the Theory and Application of Cryptology and Information Security*, Seoul, South Korea, December 4–8, 2011. *Proceedings 17*. pp. 21–40. Springer (2011)
- [4] Agrawal, S., Goyal, R., Tomida, J.: Multi-party functional encryption. In: *Theory of Cryptography: 19th International Conference, TCC 2021, Raleigh, NC, USA, November 8–11, 2021, Proceedings, Part II*. p. 224–255. Springer-Verlag, Berlin, Heidelberg (2021)
- [5] Albrecht, M.R., Benčina, B., Lai, R.W.F.: Hollow LWE: A new spin - unbounded updatable encryption from LWE and PCE. In: Fehr, S., Fouque, P. (eds.) *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques Madrid, Spain, May 4–8, 2025, Proceedings, Part VIII. Lecture Notes in Computer Science*, vol. 15608, pp. 363–392. Springer (2025)
- [6] Ayday, E., Raisaro, J.L., Hengartner, U., Molyneaux, A., Hubaux, J.P.: Privacy-preserving processing of raw genomic data. In: Garcia-Alfaro, J., Lioudakis, G., Cuppens-Boulahia, N., Foley, S., Fitzgerald, W.M. (eds.) *Data Privacy Management and Autonomous Spontaneous Security*. pp. 133–147. Springer Berlin Heidelberg, Berlin, Heidelberg (2014)
- [7] Bakas, A., Michalas, A., Dimitriou, T.: Private lives matter: A differential private functional encryption scheme. In: *Proceedings of the Twelfth ACM Conference on Data and Application Security and Privacy*. p. 300–311. CODASPY '22, Association for Computing Machinery, New York, NY, USA (2022)
- [8] Baltico, C.E.Z., Catalano, D., Fiore, D., Gay, R.: Practical functional encryption for quadratic functions with applications to predicate encryption. In: Katz, J., Shacham, H. (eds.) *Advances in Cryptology – CRYPTO 2017*. pp. 67–98. Springer International Publishing, Cham (2017)
- [9] Boneh, D., Lewi, K., Montgomery, H., Raghunathan, A.: Key homomorphic prfs and their applications. In: Canetti, R., Garay, J.A. (eds.) *Advances in Cryptology – CRYPTO 2013*. pp. 410–428. Springer Berlin Heidelberg, Berlin, Heidelberg (2013)
- [10] Boneh, D., Sahai, A., Waters, B.: Functional encryption: Definitions and challenges. In: *Theory of Cryptography: 8th Theory of Cryptography Conference, TCC 2011, Providence, RI, USA, March 28–30, 2011. Proceedings 8*. pp. 253–273. Springer (2011)
- [11] Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. In: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*. p. 309–325. ITCS '12, Association for Computing Machinery, New York, NY, USA (2012)
- [12] Cini, V., Ramacher, S., Slamanig, D., Striecks, C., Tairi, E.: (inner-product) functional encryption with updatable ciphertexts. *J. Cryptol.* **37**(1) (Dec 2023)
- [13] Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. *IACR Cryptol. ePrint Arch.* **2012**, 144 (2012)
- [14] Fang, S., Yang, S., Zhang, Y.: Inner product encryption from ring learning with errors. *Cybersecurity* **3**, 1–11 (2020)
- [15] Goldwasser, S., Gordon, S.D., Goyal, V., Jain, A., Katz, J., Liu, F.H., Sahai, A., Shi, E., Zhou, H.S.: Multi-input functional encryption. In: Nguyen, P.Q., Oswald, E. (eds.) *Advances in Cryptology – EUROCRYPT 2014*. pp. 578–602. Springer Berlin Heidelberg, Berlin, Heidelberg (2014)
- [16] Hasan, M.Z., Mahdi, M.S.R., Sadat, M.N., Mohammed, N.: Secure count query on encrypted genomic data. *Journal of Biomedical Informatics* **81**, 41–52 (2018)
- [17] Li, Y., Wei, J., Guo, F., Susilo, W., Chen, X.: Robust decentralized multi-client functional encryption: Motivation, definition, and inner-product constructions. In: Guo, J., Steinfeld, R. (eds.) *Advances in Cryptology – ASIACRYPT 2023*. pp. 134–165. Springer Nature Singapore, Singapore (2023)
- [18] Libert, B., T̃ĩu, R.: Multi-client functional encryption for linear functions in the standard model from lwe. In: Galbraith, S.D., Moriai, S. (eds.) *Advances in Cryptology – ASIACRYPT 2019*. pp. 520–551. Springer International Publishing, Cham (2019)
- [19] Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. *J. ACM* **60**(6) (Nov 2013)
- [20] Meers, J., Riepel, D.: Cca secure updatable encryption from non-mappable group actions. In: Saarinen, M.J., Smith-Tone, D. (eds.) *Post-Quantum Cryptography*. pp. 137–169. Springer Nature Switzerland, Cham (2024)
- [21] Mera, J.M.B., Karmakar, A., Marc, T., Soleimanian, A.: Efficient lattice-based inner-product functional encryption. In: *IACR International Conference on Public-Key Cryptography*. pp. 163–193. Springer (2022)
- [22] Miao, P., Patrnanabis, S., Watson, G.: Unidirectional updatable encryption and proxy re-encryption from ddh. In: Boldyreva, A., Kolesnikov, V. (eds.) *Public-Key Cryptography – PKC 2023*. pp. 368–398. Springer Nature Switzerland, Cham (2023)
- [23] Mono, J., Marcolla, C., Land, G., Güneysu, T., Aaraj, N.: Finding and evaluating parameters for bgv. In: *Progress in Cryptology - AFRICACRYPT 2023: 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 19–21, 2023, Proceedings*. p. 370–394. Springer-Verlag, Berlin, Heidelberg (2023)
- [24] Nuoskala, C., Abdinasibfar, H., Michalas, A.: Spade: Digging into selective and partial decryption using functional encryption. In: *20th EAI International Conference on Security and Privacy in Communication Networks (SecureComm'24)*. *SecureComm 2024*, Springer Nature Switzerland, Cham (2024)
- [25] Pal, T., Dutta, R.: Attribute-based access control for inner product functional encryption from lwe. In: *International Conference on Cryptology and Information Security in Latin America*. pp. 127–148. Springer (2021)
- [26] Slamanig, D., Striecks, C.: Revisiting updatable encryption: Controlled forward security, constructions and a puncturable perspective. In: *Theory of Cryptography: 21st International Conference, TCC 2023, Taipei, Taiwan, November 29 – December 2, 2023, Proceedings, Part II*. p. 220–250. Springer-Verlag, Berlin, Heidelberg (2023)
- [27] Tomida, J.: Unbounded quadratic functional encryption and more from pairings. In: *Advances in Cryptology – EUROCRYPT 2023: 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23–27, 2023, Proceedings, Part III*. p. 543–572. Springer-Verlag, Berlin, Heidelberg (2023)